

Interfaces in Java

An interface defines what methods an object has, but not how these methods are implemented. It includes the name of the method, its parameters, return type, and what exceptions it can throw — but not the method's body, which contains the instructions for executing that method.

That is, an interface contains only the **signature** of each method. You can declare an interface this way in Java:

```
1: import java.util.List;
2:
3: public interface Register {
4:     public void addName(String name) throws AlreadyPresentException;
5:
6:     public List<String> getNames();
7: }
```

An interface is not the same as a class:

- It's declared as an interface instead of a class.
- Instead of braces containing a method body, there is a semicolon `;` after each method signature.
- An interface cannot have instance fields, since these are not part of how a class should be used by other classes.

This interface declares that a Register can be used by calling its addName method or by calling its getName method. However, the interface doesn't say what these methods should do — it's up to the class to decide that.

The AttendanceRegister class has these methods with these signatures, but it also has **implementations** of the methods. Therefore we can say that the AttendanceRegister class "implements" the Register interface:


```
1: import java.util.*;
2:
3: public class AttendanceRegister implements Register {
4:     private final List<String> names = new ArrayList<>();
5:
6:     @Override
7:     public void addName(String name) throws AlreadyPresentException {
8:         if(names.contains(name)) {
9:             throw new AlreadyPresentException(name);
10:        }
11:        names.add(name);
12:    }
13:
14:    @Override
15:    public List<String> getNames() {
16:        return Collections.unmodifiableList(names);
17:    }
18: }
```

The `implements` keyword indicates that the `AttendanceRegister` class contains implementations for the methods in the `Register` interface.

The `@Override` annotation indicates the method “overrides” a method declared somewhere else (in this case, the `Register` interface) – this annotation is optional, but helps make the code easier to read and understand.

Thinking of message passing, `Register` defines what messages the class must accept and what kinds of response it can give; `AttendanceRegister` additionally defines how those responses are computed.

Multiple implementations

Although `Register` is the interface for the `AttendanceRegister` class, it is possible for other classes to have the same public methods with the same signatures. In this case, those classes can also implement the `Register` interface.

For example, the `SoloRegister` class models a register for a one-on-one tutorial which only one student is expected to attend:

```
1: import java.util.*;
2:
3: public class SoloRegister implements Register {
4:     private String name = null;
5:
6:     @Override
7:     public void addName(String name) throws AlreadyPresentException {
8:         if(this.name == null) {
9:             this.name = name;
10:        } else if(this.name.equals(name)) {
11:            throw new AlreadyPresentException(name);
12:        } else {
13:            String message = "Can't add second name: " + this.name + ", " + name;
14:            throw new IllegalStateException(message);
15:        }
16:    }
17:
18:    @Override
19:    public List<String> getNames() {
20:        if(name == null) {
21:            return Collections.emptyList();
22:        } else {
23:            return Collections.singletonList(name);
24:        }
25:    }
26: }
```

The `SoloRegister` class is implemented quite differently to the `AttendanceRegister` class: since only one name needs to be stored, there is a field for a single name rather than a list of names. Since the `getNames` method must return a list, this implementation uses methods from the `Collections` helper class to create one.

Interfaces are types

A `Register` can be used in two ways: either by calling its `addName` method, or by calling its `getNames` method. An `AttendanceRegister` can be used in both of those ways, so it is possible to use an `AttendanceRegister` as a `Register`:

```
Register register = new AttendanceRegister();
register.addName("Alice");
register.addName("Bob");
register.addName("Charles");
register.getNames().toString()
    "[Alice, Bob, Charles]" (String)
```

Similarly, a `SoloRegister` can be used as a `Register`:

```
Register solo = new SoloRegister();
solo.addName("Alice");
solo.getNames().toString()
    "[Alice]" (String)
```

In these examples, `Register` is the **type** of the variable, but the object is created using the `AttendanceRegister` constructor or the `SoloRegister` constructor. This works because both classes implement the `Register` interface.

We have already seen another situation where this is possible:

```
import java.util.*;
List<String> names = new ArrayList<String>();
```

The variable's type is `List<String>`, but the object is created using the `ArrayList<String>` constructor. This works because `List` is an interface, and the `ArrayList` class implements this interface.

This makes sense: `List` is an [abstract data type](#), which means it specifies what methods a list has but not how those methods are implemented. The `ArrayList` and `LinkedList` data structures are implementations of the `List` interface.

Polymorphism

Since one method can have more than one implementation, when we write code to call this method, we don't necessarily know which implementation will be executed:

```
1: public class RegisterExample {
2:     public static String getExampleNames(Register register) {
3:         try {
4:             register.addName("Alice");
5:             register.addName("Bob");
6:             register.addName("Charles");
7:             return register.getNames().toString();
8:         } catch (AlreadyPresentException ex) {
9:             // should never happen; convert to unchecked exception
10:            throw new RuntimeException(ex);
11:        }
12:    }
13: }
```

This method takes a Register as its parameter, which means the method could receive either an AttendanceRegister object or a SoloRegister object.⁽¹⁾

When we write this code, we know we will call the addName method on the register object. However, we don't know what class register is, so we don't know which implementation of the addName method will be executed.

This is an example of **polymorphism**: there is more than one class which the parameter register could have, and the call to the addName method executes different instructions depending on which class. The getExampleNames method is “polymorphic”.

Thinking about message passing, we know that register will accept the addName message, but we don't know how it will handle that message. Importantly, different implementations can result in different behaviour:

```
RegisterExample.getExampleNames(new AttendanceRegister())
    "[Alice, Bob, Charles]" (String)
RegisterExample.getExampleNames(new SoloRegister())
Exception: java.lang.IllegalStateException (Can't add second name: Alice, Bob)
    at SoloRegister.addName(SoloRegister.java:14)
```

The getExampleNames method has a different result the second time, because the SoloRegister class throws an exception but the AttendanceRegister class doesn't.